FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERIA MECANICA Y ELECTRICA.

Actividad: CONEXIÓN A LA BASE DE DATOS.

MATERIA: INGENIERIA DE SOFTWARE.

PROFESORA: ALICIA YESENIA LOPEZ SANCHEZ

Nelson Tadeo Muñoz Flores	2227010	IIA
Abiud Emiliano Pérez Palos	2063724	IIA

FECHA: 26/10/26

GRUPO: 001

San Nicolas de los Garza/Nuevo León en FIME-Ciudad Universitaria.

Este reporte documenta un sistema web gerencial con inicio de sesión, control de inventario, registro de ingresos y merma, y un simulador de punto de venta conectado al inventario. La información se guarda en una base de datos de Firebase Firestore y el código está escrito en JavaScript.

Elemento	Tecnología
Interfaz	HTML y CSS
Lógica	JavaScript (sin frameworks)
Base de datos	Firebase Firestore
Editor y servidor local	Visual Studio Code con Live Server
Control de versiones	Git y GitHub

Interfaces del sistema

El sistema tiene cuatro pantallas principales y el módulo de inventario se divide en cinco pestañas. Todas comparten los colores de la marca (rojo y amarillo) y se adaptan a pantallas chicas.

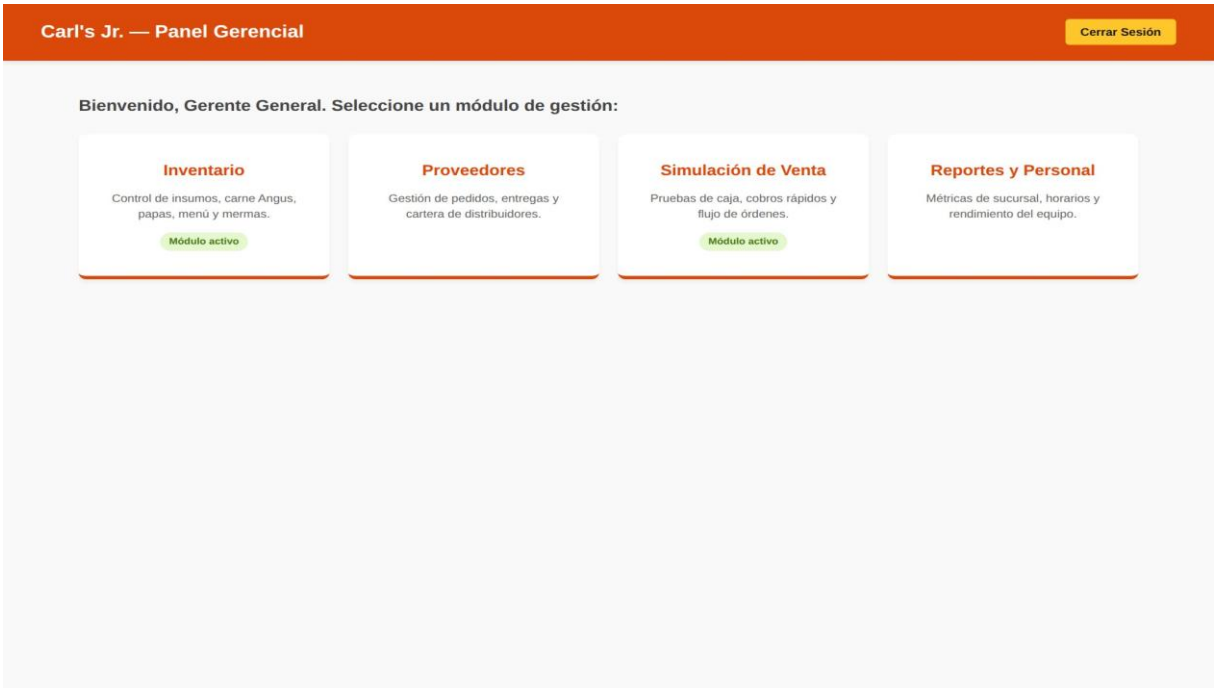
1. Inicio de sesión

Pantalla de acceso gerencial. Valida el usuario y la contraseña contra la lista de usuarios autorizados; si no coinciden muestra un mensaje de error. La sesión se conserva al recargar la página y se cierra con el botón Cerrar Sesión.



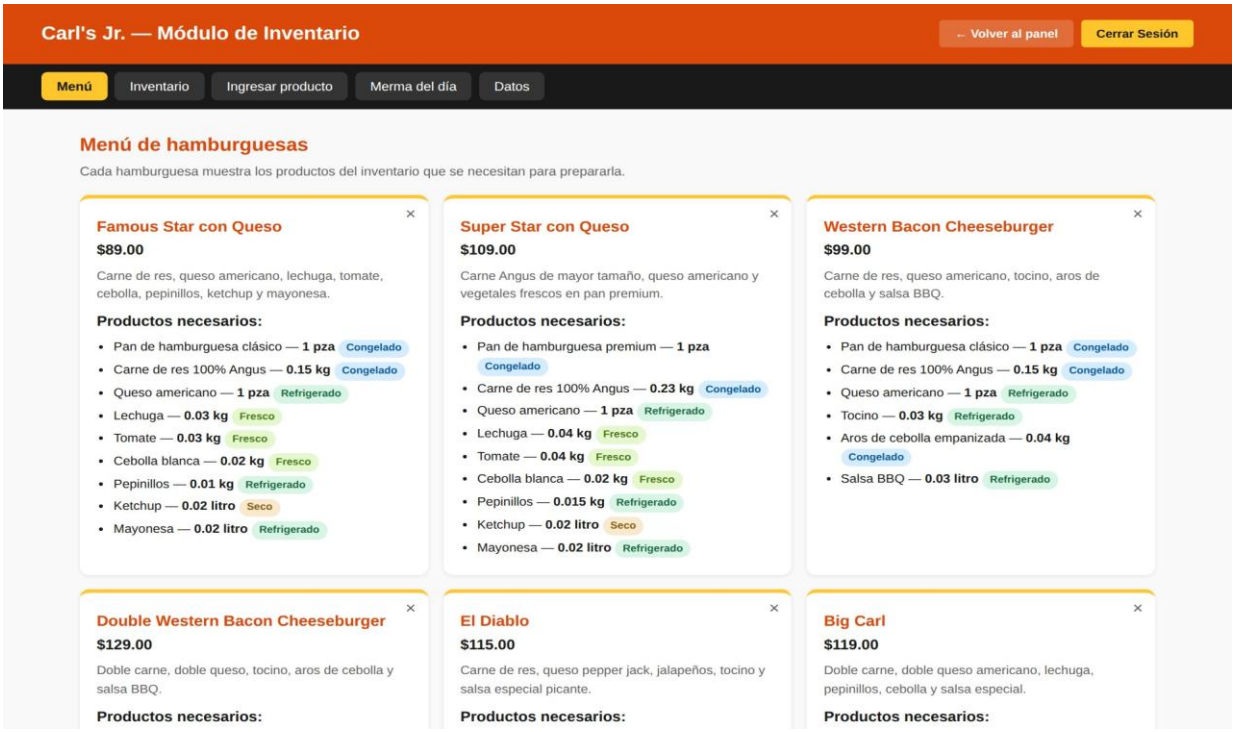
2. Panel gerencial

Menú principal con cuatro módulos: Inventario y Simulación de Venta están activos; Proveedores y Reportes y Personal muestran su descripción y quedan pendientes de desarrollo. Saluda al usuario que inició sesión.



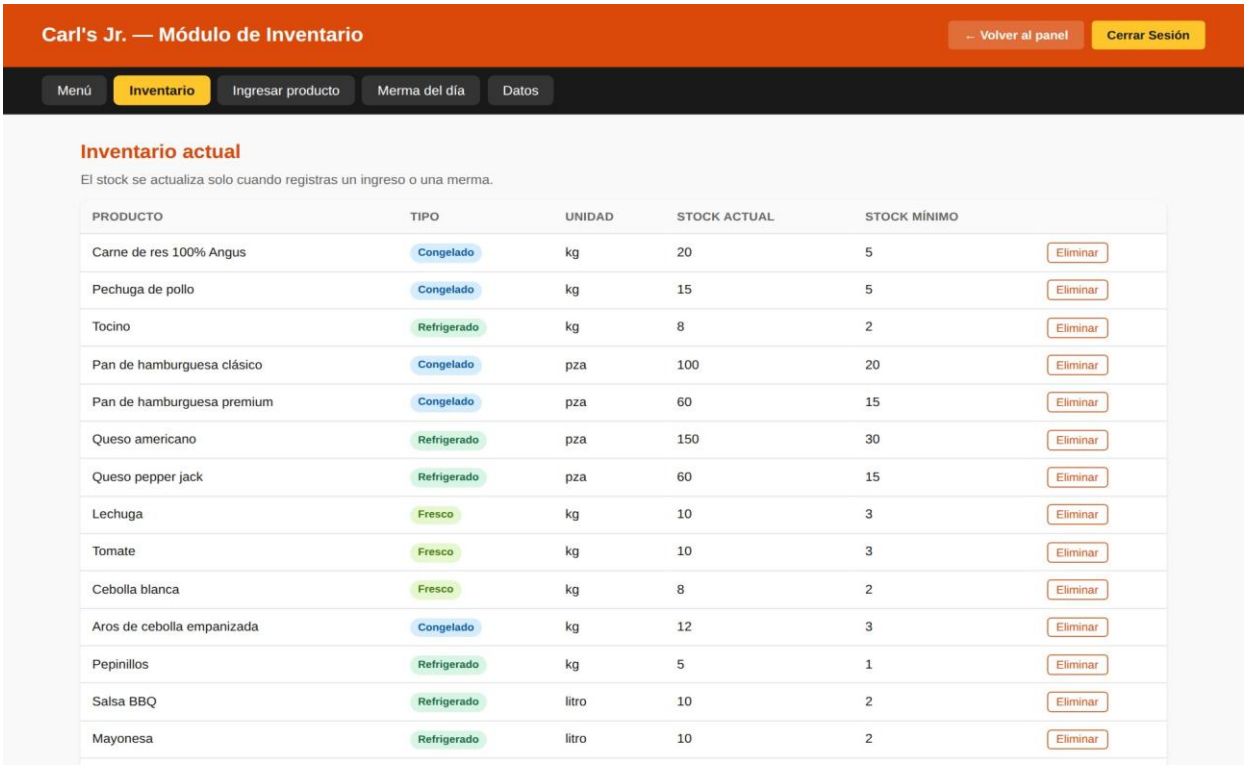
3. Inventario: pestaña Menú

Muestra cada hamburguesa con su precio, su descripción y los productos del inventario que necesita para prepararla (la receta). Abajo hay un formulario para agregar hamburguesas nuevas eligiendo sus ingredientes.



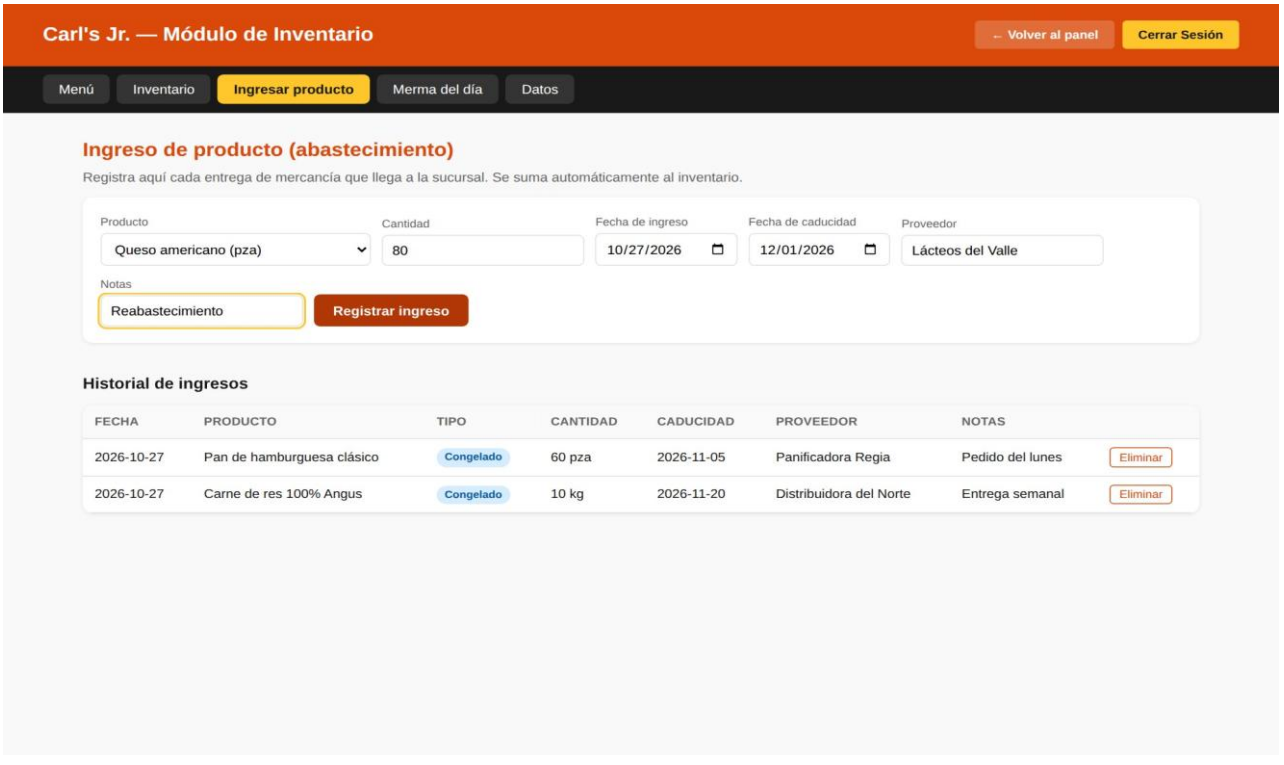
4. Inventario: pestaña Inventario

Catálogo de productos con tipo (congelado, refrigerado, fresco o seco), unidad de medida, stock actual y stock mínimo. El stock que queda por debajo del mínimo se marca en rojo.
Incluye un formulario para dar de alta productos nuevos.



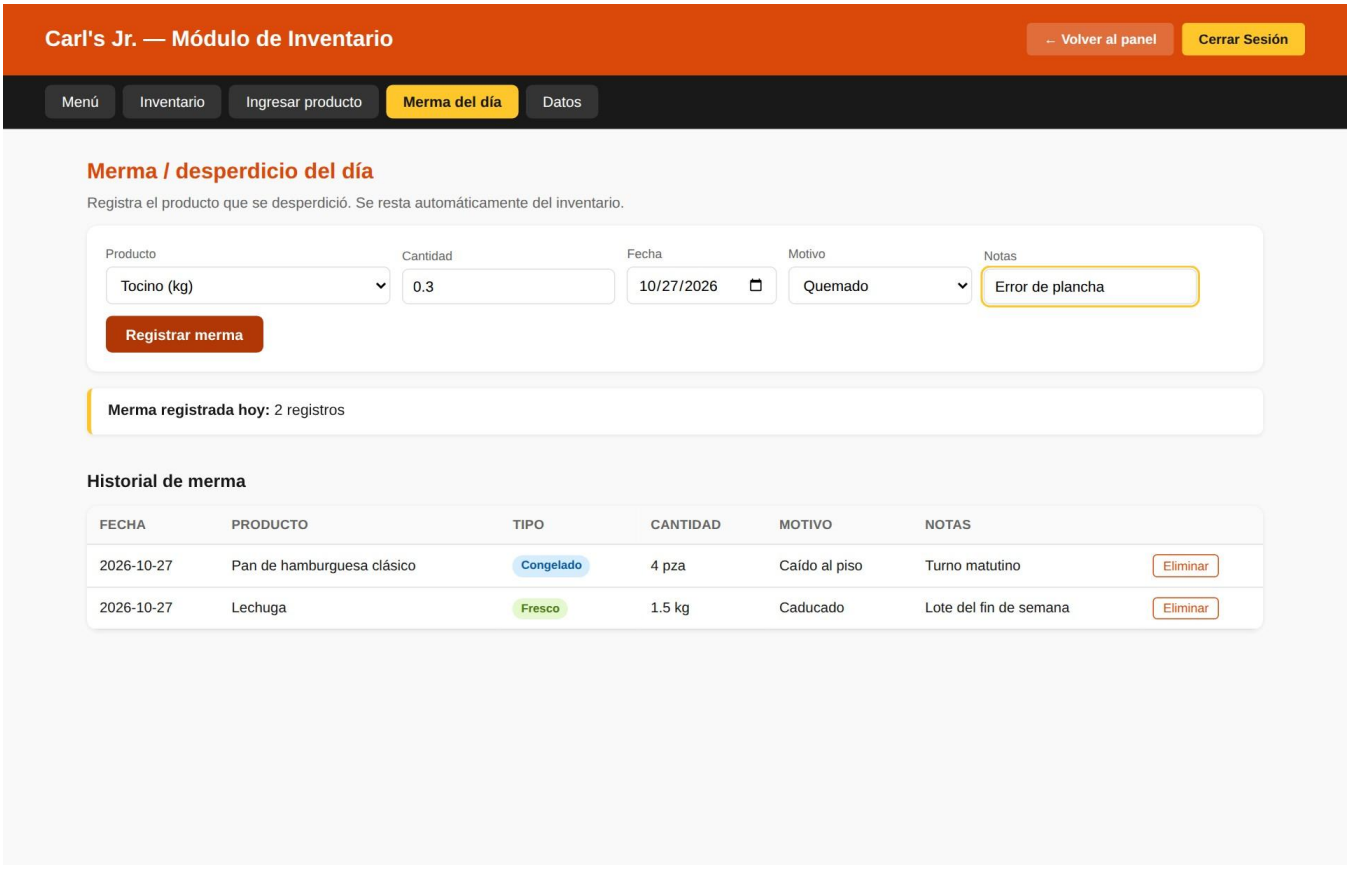
5. Inventario: pestaña Ingresar producto

Registro manual del abastecimiento: producto, cantidad, fecha de ingreso, fecha de caducidad, proveedor y notas. Cada ingreso se suma automáticamente al stock y queda en el historial de ingresos.



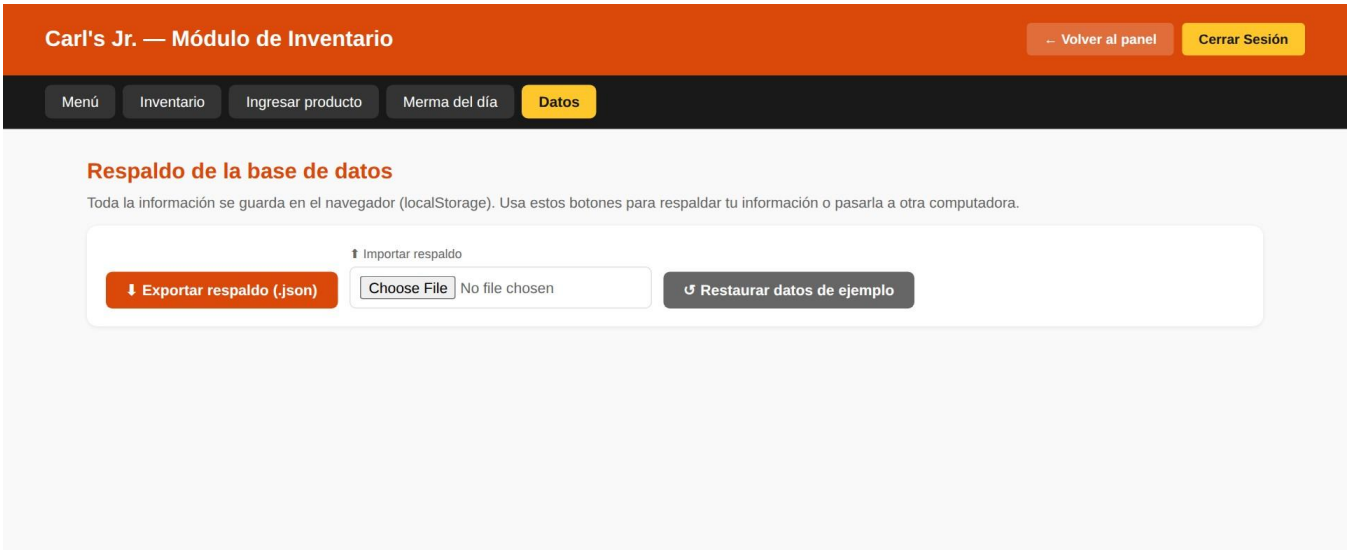
6. Inventario: pestaña Merma del día

Registro manual del desperdicio: producto, cantidad, fecha, motivo (caducado, dañado, quemado, entre otros) y notas. Cada registro se resta del stock, el sistema rechaza mermas mayores al stock disponible y un contador muestra los registros de hoy.



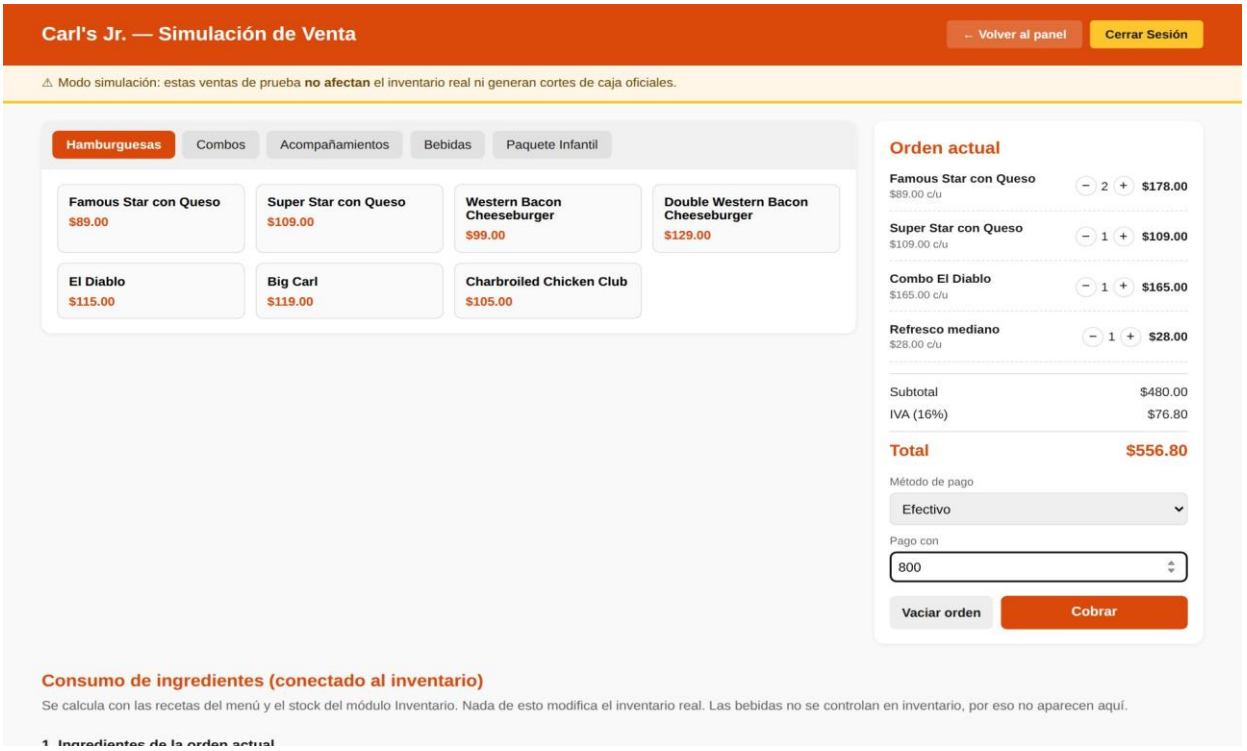
7. Inventario: pestaña Datos

Respaldo de la información: exporta todo a un archivo JSON, importa un respaldo y restaura los datos de ejemplo.



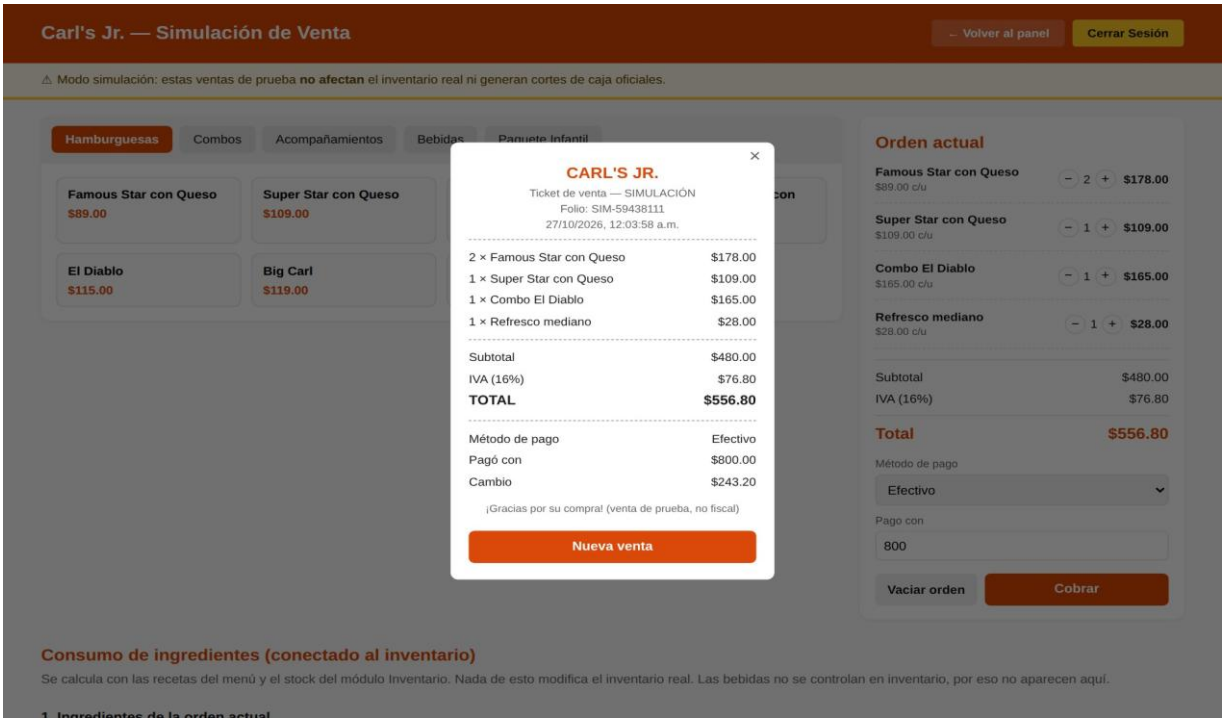
8. Simulación de venta

Punto de venta de práctica. El catálogo se organiza en hamburguesas, combos, acompañamientos, bebidas y paquete infantil. La orden actual calcula subtotal, IVA del 16% y total; el pago puede ser en efectivo (con cálculo de cambio) o con tarjeta. Es un modo de simulación: no modifica el inventario real.



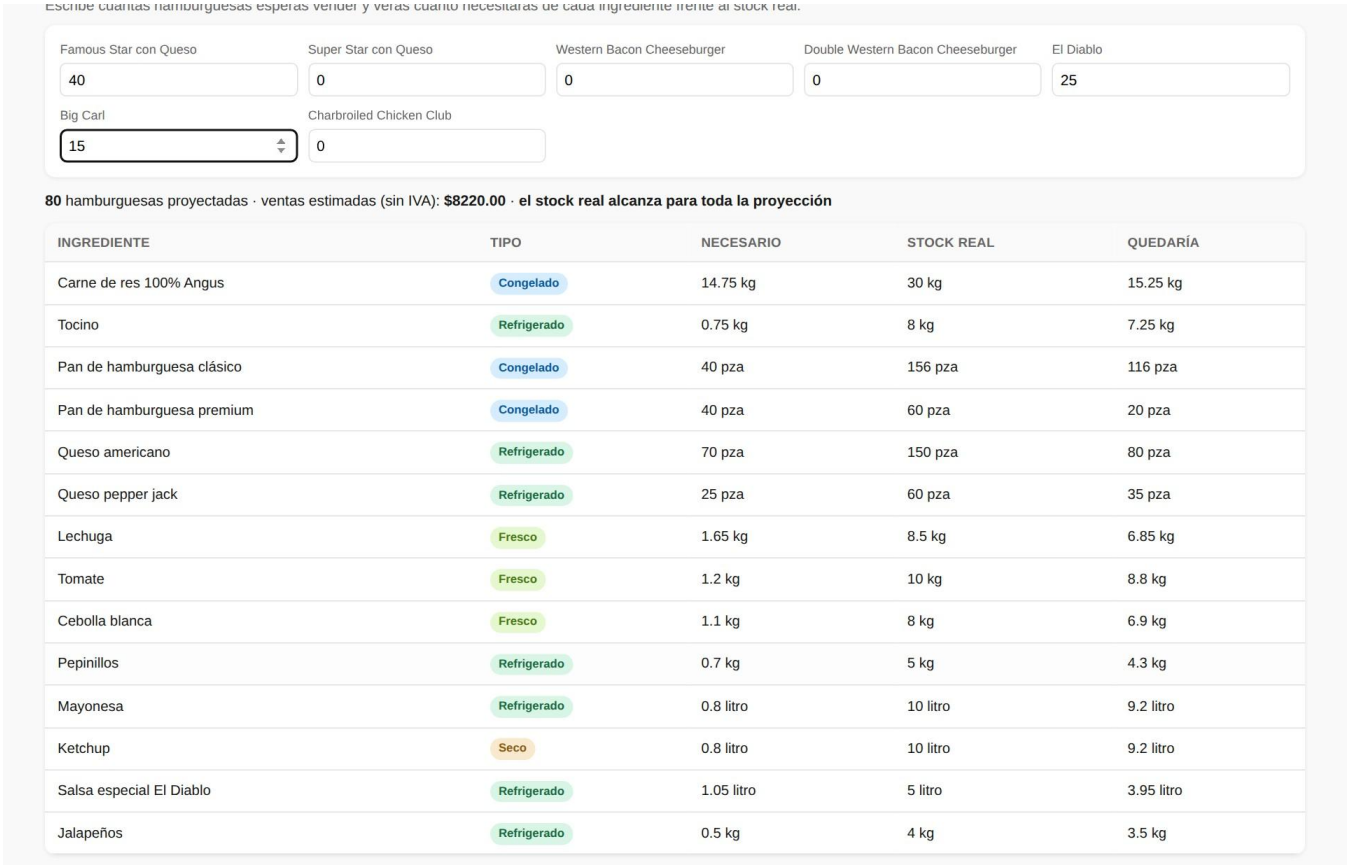
9. Ticket de venta

Ventana emergente que aparece al cobrar, con folio, fecha y hora, artículos, totales, método de pago y cambio. La venta queda en un historial de ventas de prueba.



10. Consumo de ingredientes y proyección

Sección de la simulación conectada al inventario. Calcula qué ingredientes consume la orden actual, lleva un inventario simulado que se descuenta con cada venta de prueba y proyecta cuánto se necesita de cada ingrediente según las hamburguesas que se espera vender.



INGREDIENTE	TIPO	NECESARIO	STOCK REAL	QUEDARÍA
Carne de res 100% Angus	Congelado	14.75 kg	30 kg	15.25 kg
Tocino	Refrigerado	0.75 kg	8 kg	7.25 kg
Pan de hamburguesa clásico	Congelado	40 pza	156 pza	116 pza
Pan de hamburguesa premium	Congelado	40 pza	60 pza	20 pza
Queso americano	Refrigerado	70 pza	150 pza	80 pza
Queso pepper jack	Refrigerado	25 pza	60 pza	35 pza
Lechuga	Fresco	1.65 kg	8.5 kg	6.85 kg
Tomate	Fresco	1.2 kg	10 kg	8.8 kg
Cebolla blanca	Fresco	1.1 kg	8 kg	6.9 kg
Pepinillos	Refrigerado	0.7 kg	5 kg	4.3 kg
Mayonesa	Refrigerado	0.8 litro	10 litro	9.2 litro
Ketchup	Seco	0.8 litro	10 litro	9.2 litro
Salsa especial El Diablo	Refrigerado	1.05 litro	5 litro	3.95 litro
Jalapeños	Refrigerado	0.5 kg	4 kg	3.5 kg

La base de datos

La información del sistema se guarda en Firebase Firestore, una base de datos en la nube de Google organizada en colecciones de documentos. Se eligió porque funciona directo desde el navegador, sin servidor propio, y permite ver los datos en una consola web y sincronizarlos en tiempo real entre varias computadoras.

Creación de la base de datos

1. Crear un proyecto en la consola de Firebase (console.firebase.google.com). ID del proyecto: [ID de tu proyecto de Firebase].
2. En Firestore Database, crear la base de datos en modo de prueba y elegir la ubicación.
3. Registrar una app web en la configuración del proyecto y copiar los seis valores de configuración (apiKey, authDomain, projectId, storageBucket, messagingSenderId y appId).
4. Pegar esos valores en el archivo js/datos/config-firebase.js del sistema.
5. Publicar las reglas de seguridad de Firestore (en modo de prueba permiten leer y escribir).

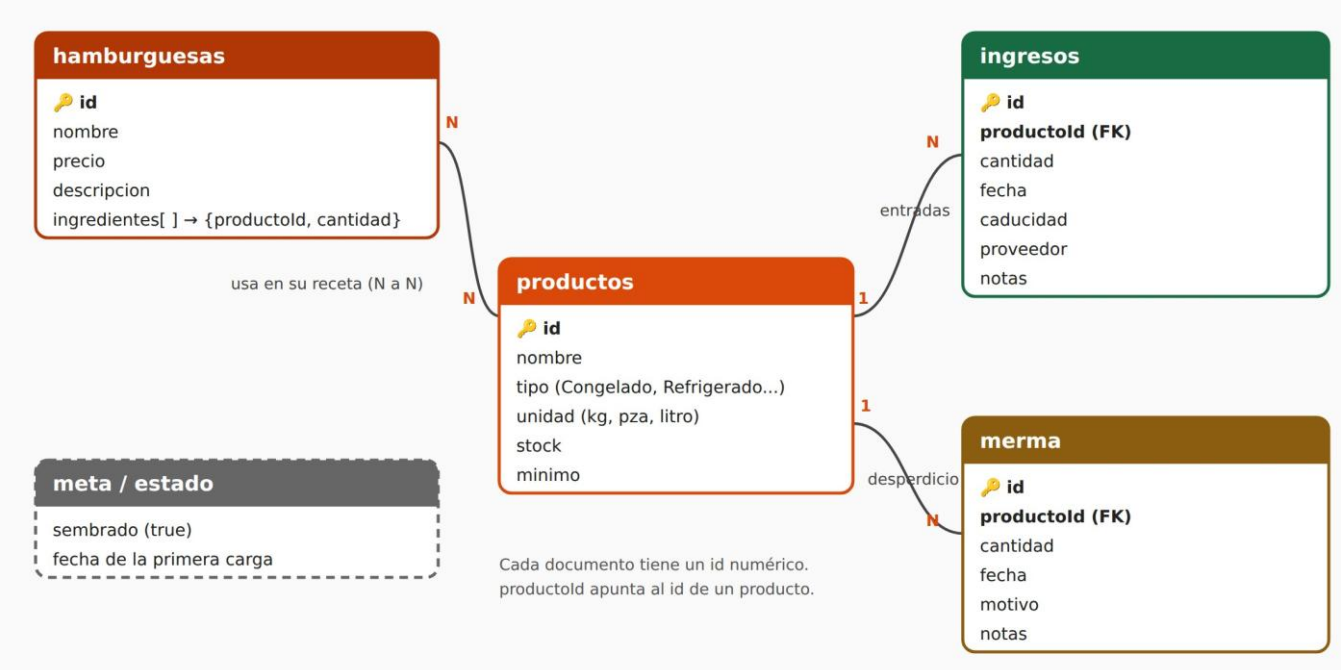
6. Abrir el sistema con Live Server. La primera vez, el propio sistema crea las colecciones y sube los productos y el menú iniciales.

Mientras la configuración no esté pegada, o si no hay internet, el sistema funciona en modo local (guarda en el navegador) y lo avisa con una etiqueta en pantalla.

Colecciones y campos

Colección	Qué guarda	Campos de cada documento
productos	Insumos del inventario	id, nombre, tipo, unidad, stock, minimo
hamburguesas	Menú y receta de cada hamburguesa	id, nombre, precio, descripcion, ingredientes (lista de productoId y cantidad)
ingresos	Cada abastecimiento de producto	id, productoId, cantidad, fecha, caducidad, proveedor, notas
merma	Cada desperdicio registrado	id, productoId, cantidad, fecha, motivo, notas
meta	Control interno de la primera carga	documento estado: sembrado, fecha

Relaciones entre colecciones



Los ingresos y la merma apuntan a un producto mediante productoId, y cada hamburguesa guarda en su receta los productoId de los insumos que usa.

Consistencia de los datos

- Al registrar un ingreso, el stock del producto se incrementa; al registrar una merma, se decrementa. El stock se envía a Firestore como incremento, para que dos computadoras que muevan el mismo producto no se pisen.
- El sistema escucha las cuatro colecciones en tiempo real: si otra computadora cambia un dato, la pantalla se actualiza sola.
- Las ventas simuladas y el inventario simulado no se guardan en Firestore: se quedan en el navegador para que la práctica no modifique los datos reales.

Captura de la consola de Firebase

[Inserta aquí la captura de Firestore Database, pestaña Datos, mostrando las colecciones productos, hamburguesas, ingresos, merma y meta de tu proyecto.]

Código fuente

El código está dividido en archivos pequeños, uno por módulo: css/ (estilos), vistas/ (HTML de cada pantalla) y js/ (lógica), que a su vez se separa en datos/ (base de datos), nucleo/ (sesión y navegación), inventario/ y ventas/. Primero se muestra la conexión a la base de datos y después la funcionalidad principal.

Conexión a la BD: configuración de Firebase

Archivo js/datos/config-firebase.js. Guarda los datos del proyecto de Firebase y la función firebaseConfigurado(), que comprueba que ya se pegaron. La captura muestra marcadores PEGA_AQUI en lugar de las claves reales del proyecto.

js/datos/config-firebase.js líneas 1–20 — Datos de conexión al proyecto de Firebase

```
/* ============================================================================
   js/datos/config-firebase.js — CONFIGURACIÓN DE TU PROYECTO DE FIREBASE
   ----------------------------------------------------------------------------
   Pega aquí los datos que te da Firebase (ver GUIA-FIREBASE.md, paso 3).
   Mientras tenga el texto "PEGA_AQUI", el sistema funciona en MODO LOCAL
   (localStorage del navegador) y avisa con una etiqueta abajo a la derecha.
   ============================================================================ */

const FIREBASE_CONFIG = {
  apiKey: "PEGA_AQUI",
  authDomain: "PEGA_AQUI",
  projectId: "PEGA_AQUI",
  storageBucket: "PEGA_AQUI",
  messagingSenderId: "PEGA_AQUI",
  appId: "PEGA_AQUI",
};

function firebaseConfigurado() {
  return Object.values(FIREBASE_CONFIG).every((v) => v && !String(v).includes("PEGA_AQUI"));
}
```

Conexión a la BD: elección del almacén

Archivo js/datos/conexion.js. Usa Firebase cuando está configurado y cargado; si no, usa el almacenamiento local del navegador. iniciarBaseDatos() conecta, muestra en pantalla dónde se guarda la información y, si Firebase falla, cae al modo local.

js/datos/conexion.js líneas 1–36 — Elige Firebase o modo local al arrancar

```
/* ============================================================================
   js/datos/conexion.js — ELIGE DÓNDE SE GUARDA LA BASE DE DATOS
   ----------------------------------------------------------------------------
   · Firebase configurado y cargado  → Firestore (nube)
   · Si no está configurado o falla  → localStorage (con aviso en pantalla)
   ============================================================================ */

let Almacen =
  firebaseConfigurado() && typeof firebase !== "undefined" ? AlmacenFirebase : AlmacenLocal;

function iniciarBaseDatos() {
  if (Almacen === AlmacenLocal) {
    estadoBD(
      "local",
      firebaseConfigurado()
        ? "Modo local: no se pudo cargar Firebase (¿hay internet?)"
        : "Modo local: Firebase sin configurar"
    );
    return AlmacenLocal.iniciar();
  }

  estadoBD("conectando", "Conectando con Firebase…");
  return Almacen
    .iniciar()
    .then(() => {
      estadoBD("firebase", "Firebase · " + FIREBASE_CONFIG.projectId,
        "https://console.firebase.google.com/project/" + FIREBASE_CONFIG.projectId + "/firestore/data");
    })
    .catch((error) => {
      console.error("No se pudo iniciar Firebase:", error);
      AlmacenFirebase.detener();
      Almacen = AlmacenLocal;
      estadoBD("error", "No se pudo conectar con Firebase (" + mensajeErrorFirebase(error) + "). Usando modo local.")
      return AlmacenLocal.iniciar();
    });
}
```

Conexión a la BD: arranque y carga inicial

Archivo js/datos/almacen-firebase.js. iniciar() inicializa Firebase y obtiene Firestore. _prepararDatos() revisa el documento meta/estado: si no existe es la primera vez, así que sube los datos iniciales y crea esa marca. Cada paso tiene un tiempo límite de 20 segundos.

js/datos/almacen-firebase.js líneas 53–91 — Conexión inicial y carga de datos de inicio

```
53  iniciar() {
54    if (!firebase.apps.length) firebase.initializeApp(FIREBASE_CONFIG);
55    this.fs = firebase.firestore();
56    COLECCIONES.forEach((c) => (this.copia[c] = {}));
57
58    return conTiempoLimite(this._prepararDatos(), 20000, "Firebase no respondió a tiempo")
59      .then(() => conTiempoLimite(this._escuchar(), 20000, "Firebase no entregó los datos a tiempo"));
60  },
61
62  detener() {
63    this.suscripciones.forEach((cancelar) => cancelar());
64    this.suscripciones = [];
65  },
66
67  /* Primera vez que se usa este proyecto de Firebase: sube los datos iniciales
68     (los que ya tenías guardados en este navegador, o los de ejemplo). */
69  _prepararDatos() {
70    const metaRef = this.fs.collection("meta").doc("estado");
71    return metaRef.get().then((doc) => {
72      if (doc.exists) return;
73      let inicial;
74      try {
75        const local = localStorage.getItem(LLAVE);
76        inicial = local ? JSON.parse(local) : datosIniciales();
77      } catch (_) {
78        inicial = datosIniciales();
79      }
80      baseDatos = {
81        productos: inicial.productos || [],
82        hamburguesas: inicial.hamburguesas || [],
83        ingresos: inicial.ingresos || [],
84        merma: inicial.merma || [],
85      };
86      return this._escribirCambios().then(() =>
87        metaRef.set({ sembrado: true, fecha: new Date().toISOString() })
88      );
89    });
90  },
91
```

Conexión a la BD: lectura en tiempo real

_escuchar() abre un listener (onSnapshot) en cada una de las cuatro colecciones y mantiene una copia en memoria. Si otra computadora cambia un dato, la copia se actualiza y las pantallas se repintan.

js/datos/almacen-firebase.js líneas 93–126 — Lectura en tiempo real de las 4 colecciones

```
93   _escuchar() {
94     return new Promise((resolver, rechazar) => {
95       let pendientes = COLECCIONES.length;
96       let listo = false;
97
98       COLECCIONES.forEach((c) => {
99         let primera = true;
100        const cancelar = this.fs.collection(c).onSnapshot(
101          (snap) => {
102            const docs = snap.docs.map((d) => d.data()).sort((a, b) => a.id - b.id);
103            if (c === "productos") docs.forEach((p) => (p.stock = redondear(p.stock)));
104
105            const nueva = {};
106            docs.forEach((d) => (nueva[d.id] = canon(d)));
107            this.copia[c] = nueva;
108
109            const cambio = canon(baseDatos[c]) !== canon(docs);
110            if (cambio) baseDatos[c] = docs;
111
112            if (primera) {
113              primera = false;
114              pendientes -= 1;
115              if (pendientes === 0) { listo = true; resolver(); }
116            } else if (cambio && typeof alCambiarDatos === "function") {
117              alCambiarDatos(); // otro equipo cambió algo: repintar pantallas
118            }
119          },
120          (error) => (listo ? this.alError(error) : rechazar(error))
121        );
122        this.suscripciones.push(cancelar);
123      });
124    });
125  },
126
```

Conexión a la BD: escritura guardar() y _escribirCambios() comparan la copia en memoria con lo último que hay en Firestore y envían solo lo que cambió (altas, cambios y bajas) en lotes. Si lo único que cambió de un producto es el stock, lo envía como incremento.

js/datos/almacen-firebase.js líneas 128–178 Escritura: solo se envía lo que cambió

```
128  guardar() {
129    this._escribirCambios().catch((error) => this.alError(error));
130  },
131
132  _escribirCambios() {
133    const ops = [];
134
135    COLECCIONES.forEach((c) => {
136      const actual = {};
137      baseDatos[c].forEach((d) => (actual[d.id] = canon(d)));
138      const previa = this.copia[c];
139
140      Object.keys(actual).forEach((id) => {
141        if (previa[id] === actual[id]) return;
142        const ref = this.fs.collection(c).doc(String(id));
143        const nuevo = JSON.parse(JSON.stringify(baseDatos[c].find((d) => String(d.id) === id)));
144
145        if (previa[id] === undefined) {
146          ops.push((lote) => lote.set(ref, nuevo));
147          return;
148        }
149        const viejo = JSON.parse(previa[id]);
150        if (c === "productos" && soloCambiaStock(viejo, nuevo)) {
151          const delta = redondear(nuevo.stock - viejo.stock);
152          ops.push((lote) => lote.update(ref, { stock: firebase.firestore.FieldValue.increment(delta) }));
153        } else {
154          ops.push((lote) => lote.set(ref, nuevo));
155        }
156      });
157
158      Object.keys(previa).forEach((id) => {
159        if (actual[id] === undefined) {
160          const ref = this.fs.collection(c).doc(String(id));
161          ops.push((lote) => lote.delete(ref));
162        }
163      });
164
165      this.copia[c] = actual; // se considera enviado desde ya (evita duplicar incrementos)
166    });
167
168    // Firestore permite 500 operaciones por lote
169    const lotes = [];
170    for (let i = 0; i < ops.length; i += 400) lotes.push(ops.slice(i, i + 400));
171    return Promise.all(
172      lotes.map((grupo) => {
173        const lote = this.fs.batch();
174        grupo.forEach((op) => op(lote));
175        return lote.commit();
176      })
177    );
178  },
```

Funcionalidad: ingreso de producto

Archivo js/datos/ingresos.js. agregarIngreso() valida el producto y la cantidad, crea el registro y suma la cantidad al stock; eliminarIngreso() revierte el movimiento. Al terminar llama a guardarDB(), que envía el cambio a Firebase.

Image — ingresos.js

Funcionalidad: inicio de sesión

Archivo js/nucleo/auth.js. Define los usuarios autorizados, valida las credenciales del formulario, guarda la sesión y abre el panel gerencial. Esta validación corre en el navegador, por lo que sirve como control de acceso básico y no como seguridad real.

Image — auth.js

Funcionalidad: consumo de ingredientes

Archivo js/ventas/consumo.js. Con las recetas guardadas en la base de datos, ingredientesDeItem() obtiene los insumos de un producto y consumoDeOrden() suma el consumo de toda la orden.

Image — consumo.js

Funcionalidad: cobro en la simulación de venta

Archivo js/ventas/cobro.js. Valida el pago, calcula el consumo de ingredientes y avisa si no alcanzan en el inventario simulado. Después arma la venta, la guarda en el historial de pruebas y descuenta solo el inventario simulado, nunca el real.

Image — cobro.js